

MiniShop — Liste complète des requêtes SQL

RESTANT À ÉCRIRE

Projet : MiniShop (SAÉ 302) — boutique en ligne pédagogique **Public** : la personne en charge des requêtes SQL de l'application PHP **Version** : 1.0 — 23 septembre 2026 **Base** : MariaDB 11.8, schéma `minishop`, charset `utf8mb4`

0. Ce qui est DÉJÀ FAIT (à ne pas réécrire)

Les objets suivants existent dans `sql/` et **ne doivent pas être recodés en PHP** — l'application ne fait que les appeler ou lire les vues. Toute réimplémentation en PHP des procédures métier est sanctionnée (pénalité MVC/SP mentionnée dans la todo list).

0.1 Tables (8) — ``sql/01_minishop_schema.sql``

`administrateur`, `client`, `categorie`, `produit`, `parametre`, `commande`, `ligne_commande`, `order_status_history`.

0.2 Vues (3) — à lire par ``SELECT`` quand elles suffisent

Vue	Usage
<code>v_etat_stock</code>	état du stock (RUPTURE/TRES_BAS/DISPONIBLE) pour chaque produit
<code>v_catalogue</code>	catalogue client (produits visible=1 + catégorie + état de stock)
<code>v_commandes_client</code>	liste des commandes avec nom du client et nombre de lignes

0.3 Procédures / fonction (17 + 1) — à appeler par ``CALL``

Procédure / Fonction	Rôle	Ecran qui l'appelle
<code>fn_param(cle, default)</code>	lit un paramètre métier (frais_port, franchise_port, tva_default...)	plusieurs écrans
<code>sp_create_account(nom, prenom, email, mdp, tel, ...)</code>	création d'un compte client	UC-04 inscription
<code>sp_get_credentials(email)</code>	récupère le hash pour vérifier le mot de passe	UC-05 connexion
<code>sp_update_client(id, nom, prenom, email, tel, adr, cp, ville)</code>	modification de compte	EF-CLI-01
<code>sp_search_products(q, cat, prix_min, prix_max, stock, tri, page, page_size)</code>	recherche/filtres/pagination catalogue	UC-01/UC-02

sp_save_product(...) / sp_delete_product(id)	CRUD produit (suppression logique si commandé)	UC-10
sp_save_category(...) / sp_delete_category(id)	CRUD catégorie	UC-11
sp_adjust_stock(id, mode, qte, motif, admin_id)	ajustement de stock (SET/DELTA)	UC-12
sp_create_order(id_client, adr) / sp_add_order_line(id, prod, qte)	création de commande à la main (démon)	— (interne, démon)
sp_create_order_from_basket(id_cli ent, adr, lignes_json)	crée une commande entière depuis le panier (transaction, décrémentation stock)	UC-07 validation
sp_confirm_order(id)	confirme la commande (calcul frais de port, montant total)	UC-07
sp_compute_shipping(id)	recalcule les frais de port (appelé par confirm)	—
sp_update_order_status(id, cible, auteur, role, commentaire)	transition de statut (matrice RB-11)	UC-14
sp_cancel_order(id, client, commentaire)	annulation client (restitution stock par trigger)	UC-09
sp_revenue_report(deb, fin)	CA par statut, panier moyen, top produits	EF-ADM-09 indicateurs

0.4 Triggers (16) — `sql/03\_minishop\_triggers.sql`

Calculent automatiquement `prix_ttc`, `total_ligne`, le snapshot `prix_unitaire`, les décrétements/restitutions de stock, la matrice de transitions RB-11, la piste `order_status_history`, les blocages RB-14 (catégorie occupée, produit commandé), l'interdiction de modifier `montant_total`, `frais_port` (RB-15).

1. Règles communes à TOUTES les requêtes écrites dans l'application

Avant toute requête, appliquer ces règles (sinon les tests de sécurité S-01...S-08 échoueront) :

1. **Toujours utiliser PDO avec requêtes préparées** `$pdo->prepare(...)` + `execute(...)`.

Jamais de concaténation de variables dans la chaîne SQL.

1. **Liste blanche pour les tris et les filtres** : `tri`, `sens`, `colonne de tri` doivent être vérifiés

côté PHP avant d'être injectés dans un `ORDER BY` (les clauses ORDER BY ne peuvent pas être paramétrées).

1. **Noms de colonnes en backticks** si nécessaire, mais jamais de noms dynamiques non validés.

2. **Aucune valeur numérique dans le SQL** pour les seuils métiers (frais de port, franchise,

TVA,

pagination par défaut) : les lires via `fn_param(...)` ou la table `parametre`.

1. **Toute écriture métier passe par une procédure** (voir §0.3). Seules des lectures `SELECT` et des

écritures purement techniques (mise à jour de `derniere_connexion`, blocage d'admin, etc.) sont autorisées en direct.

1. **Pagination bornée** : `page_size` par défaut = 12, maximum 60. `OFFSET` calculé comme $(page - 1) * page_size$ après avoir casté `page` en entier ≥ 1 .

1. **Les montants sont renvoyés en tant que chaînes/décimales** par PDO pour éviter les erreurs

d'arrondi float — ne caster ni en `(float)` ni en `(int)`.

2. Requêtes à écrire, par module

Légende des colonnes

- **Id** : référence EF/SEC/ENF du CDC

- **Écran/Référence** : route et nom du cas d'usage

- **Question à répondre** : ce que la requête doit ramener ou faire, en langage métier

- **Type** : `SELECT`, `INSERT`, `UPDATE`, `CALL proc`, `UPDATE derniere_connexion...`

- **Tables/Vues** : tables ou vues mises en jeu

- **Filtres / contraintes** : clauses `WHERE` / `JOIN` obligatoires

- **Tri / Pagination** : `ORDER BY` / `LIMIT`

- **Retour attendu** : colonnes ou lignes que la requête doit fournir à la vue

2.1 Module Commun (layout, en-tête, pages publiques)

Q-COM-01 — Lire un paramètre métier

Champ	Valeur
Id	ENF-14, RB-15
Écran	tous les écrans (footer, panier, commande)
Question	Quelle est la valeur actuelle du paramètre <code>frais_port</code> ? de <code>franchise_port</code> ? de <code>tva_defaut</code> ?
Type	<code>SELECT</code> simple (ou utiliser <code>SELECT fn_param('frais_port', '4.90') AS v</code>)
Tables	<code>parametre</code>
Filtres	<code>WHERE cle = ?</code>

Tri	—
Retour	une ligne, une colonne valeur (DECIMAL 10,2 typiquement)

SQL attendu :

```
SELECT valeur FROM parametre WHERE cle = :cle;
-- ou, pour avoir une valeur par défaut si la clé manque :
SELECT fn_param(:cle, :default) AS valeur;
```

Q-COM-02 — Liste des catégories pour le menu principal

Champ	Valeur
Id	EF-VIS-01
Écran	layout principal (menu), page /categories
Question	Afficher chaque catégorie avec son nom, son slug et le nombre de produits visibles qu'elle contient.
Type	SELECT avec LEFT JOIN et COUNT
Tables	categorie, produit
Filtres	compter uniquement les produits avec visible = 1 ; les catégories sans produit visible doivent apparaître avec un compteur à 0
Tri	ORDER BY c.nom ASC
Retour	id_categorie, nom, slug, nb_produits

SQL attendu :

```
SELECT c.id_categorie, c.nom, c.slug,
       COUNT(p.id_produit) AS nb_produits
FROM   categorie c
LEFT JOIN produit p ON p.id_categorie = c.id_categorie AND p.visible = 1
GROUP BY c.id_categorie, c.nom, c.slug
ORDER BY c.nom ASC;
```

Q-COM-03 — Vérifier qu'un administrateur est actif (middleware)

Champ	Valeur
Id	SEC-04, EF-ADM-10
Écran	middleware d'authentification back-office
Question	L'administrateur connecté (id en session) existe-t-il, est-il actif = 1 et quel est son rôle ?
Type	SELECT pour contrôle de session
Tables	administrateur
Filtres	WHERE id_admin = ? AND actif = 1
Retour	id_admin, nom, prenom, email, role (une seule ligne)

```
SELECT id_admin, nom, prenom, email, role
FROM   administrateur
WHERE  id_admin = :id AND actif = 1
LIMIT 1;
```

Q-COM-04 — Mettre à jour l'horodatage de dernière connexion (client)

Champ	Valeur
Id	EF-VIS-07
Écran	POST /connexion après password_verify réussi
Question	Enregistrer la date/heure actuelle dans dernière_connexion du client qui vient de se connecter.
Type	UPDATE
Tables	client
Filtres	WHERE id_client = ?
Retour	— (mise à jour uniquement)

```
UPDATE client
  SET dernière_connexion = CURRENT_TIMESTAMP
 WHERE id_client = :id;
```

Q-COM-05 — Mettre à jour l'horodatage de dernière connexion (admin)

Champ	Valeur
Id	EF-ADM-10
Écran	POST /admin/connexion
Question	Idem Q-COM-04 pour la table administrateur.
Type	UPDATE
Tables	administrateur

```
UPDATE administrateur
  SET dernière_connexion = CURRENT_TIMESTAMP
 WHERE id_admin = :id;
```

2.2 Module Visiteur — Catalogue, recherche, fiche produit (UC-01, UC-02, UC-03)**Q-VIS-01 — Liste des produits du catalogue (sans recherche ni filtre)**

Champ	Valeur
Id	EF-VIS-01, EF-VIS-05
Écran	GET /catalogue
Question	Retourner une page de produits visibles avec leur catégorie et leur état de stock.
Type	SELECT sur vue + LIMIT/OFFSET
Tables/Vues	préférer v_catalogue (joint déjà catégorie + état), ou produit + categorie + v_etat_stock
Filtres	v_catalogue filtre déjà visible = 1 ; pas de filtre supplémentaire sur la page d'accueil

Tri	ORDER BY p.date_creation DESC par défaut ; la liste blanche des tris autorisés comprend : nom, prix_ttc ASC, prix_ttc DESC, date_creation DESC
Pagination	LIMIT :page_size OFFSET :offset (page_size par défaut 12, max 60)
Retour	id_produit, reference, slug, nom, prix_ttc, stock, categorie, categorie_slug, etat, image_url
Note : Également besoin de compter le total (Q-VIS-02) pour afficher « page n/N ».	

```
SELECT id_produit, reference, slug, nom, prix_ht, tva, prix_ttc, stock,
       categorie, categorie_slug, etat, image_url
FROM v_catalogue
ORDER BY date_creation DESC -- tri validé par la liste blanche PHP
LIMIT :limit OFFSET :offset;
```

Q-VIS-02 — Compter le nombre total de produits du catalogue (pour pagination)

Champ	Valeur
Id	EF-VIS-05, ENF-04
Écran	catalogue et recherche
Question	Combien de produits correspondent aux critères courants (utile pour calculer le nombre de pages) ?
Type	SELECT COUNT(*)
Tables/Vues	v_catalogue (avec les mêmes clauses WHERE qu'en Q-VIS-03 quand filtres actifs)

```
SELECT COUNT(*) AS total FROM v_catalogue;
-- avec filtres : WHERE id_categorie = :cat AND prix_ttc BETWEEN :min AND :max ...
```

Q-VIS-03 — Catalogue filtré + trié (catégorie, prix, stock)

Champ	Valeur
Id	EF-VIS-03
Écran	GET /catalogue?cat=&prix_min=&prix_max=&stock=1&tri=
Question	Retourner la page de produits correspondant au(x) filtre(s) cumulés.
Type	SELECT avec WHERE dynamique
Filtres	v_catalogue.id_categorie = :cat si catégorie précisée · prix_ttc BETWEEN :prix_min AND :prix_max si les deux bornes sont renseignées · stock > 0 si stock=1 (disponibles uniquement)
Tri	liste blanche identique à Q-VIS-01
Retour	mêmes colonnes que Q-VIS-01

Astuce : **Alternative** : au lieu de reconstruire le WHERE à la main, il est **recommandé** d'appeler la procédure

```
sp_search_products(NULL, :cat, :prix_min, :prix_max, :stock_only, :tri, :page, :page_size)
```

(elle fait exactement ce travail). La requête ci-dessous reste à écrire pour la page `/catalogue`

qui n'a pas de terme de recherche, mais elle peut très bien appeler la procédure avec `q=NULL`.

Q-VIS-04 — Recherche plein texte

Champ	Valeur
Id	EF-VIS-02, SEC-02
Écran	GET /recherche?q=...
Question	Retourner les produits visibles dont le nom, la description ou la référence contient le terme q (insensible à la casse et aux accents).
Type	CALL sp_search_products(...) — ne pas réécrire la recherche à la main
Paramètres	q (1-120 car., trim), puis les mêmes filtres que Q-VIS-03, tri, page, page_size
Retour	la procédure retourne deux jeux de résultats : 1) la liste paginée (colonnes idem Q-VIS-01), 2) une ligne total_trouves

Appel PDO attendu :

```
$stmt = $pdo->prepare("CALL sp_search_products(:q, :cat, :pmin, :pmax, :stock, :tri, :page, :psize)");
$stmt->execute([...]);
$rows = $stmt->fetchAll();
$stmt->nextRowset();
$total = $stmt->fetchColumn();
```

Q-VIS-05 — Fiche produit détaillée

Champ	Valeur
Id	EF-VIS-04
Écran	GET /produit/{slug}
Question	Afficher toutes les informations d'un produit visible à partir de son slug. Si le produit est masqué (visible=0) ou n'existe pas, renvoyer NULL pour que le contrôleur émette un 404.
Type	SELECT sur v_catalogue (ou produit + categorie + v_etat_stock)
Filtres	WHERE slug = :slug ; ne pas oublier que v_catalogue ne contient que visible = 1 (donc un produit masqué ne sera pas trouvé, ce qui est le comportement 404 voulu)
Retour	toutes les colonnes produit + categorie, categorie_slug, etat, une information disponible (stock > 0) dérivable de stock

```
SELECT id_produit, reference, slug, nom, description,
```

```

    prix_ht, tva, prix_ttc, stock,
    categorie, categorie_slug, etat, image_url
FROM v_catalogue
WHERE slug = :slug
LIMIT 1;

```

Astuce : Si vous avez besoin de la fiche **même quand `visible = 0`** (page d'admin), utiliser directement

`produit JOIN categorie JOIN v_etat_stock` sans le filtre `visible`.

Q-VIS-06 — Produits d'une catégorie pour lister une catégorie seule

Champ	Valeur
Id	EF-VIS-03 (variante)
Écran	GET /categorie/{slug}
Question	Même liste paginée que Q-VIS-01, mais filtrée par le slug de la catégorie (et non son id).
Type	SELECT v_catalogue JOIN categorie ou ajouter categorie_slug = :slug dans Q-VIS-03

Q-VIS-07 — Vérifier l'existence d'un produit pour l'ajout panier (avant CALL)

Champ	Valeur
Id	EF-CLI-03, RB-03, RB-18
Écran	POST /panier/ajouter
Question	Le produit id_produit existe-t-il, est-il visible, et quel est son stock actuel ? Utilisée côté PHP avant d'inscrire dans la session (le panier est en session, pas en base).
Type	SELECT
Tables	produit
Filtres	WHERE id_produit = ? AND visible = 1

```

SELECT id_produit, nom, slug, prix_ttc, stock
FROM produit
WHERE id_produit = :id AND visible = 1
LIMIT 1;

```

2.3 Module Compte client — inscription, connexion, profil (UC-04, UC-05)

Q-CLI-01 — Créer un compte client

Champ	Valeur
Id	EF-VIS-06, RB-01, RB-12, RB-13
Écran	POST /inscription

Question	Créer un nouveau client après validation des champs.
Type	CALL sp_create_account(...) — ne jamais faire d'INSERT INTO client direct
Paramètres	nom, prenom, email, mot_de_passe_hash (produit par password_hash(\$mdp, PASSWORD_BCRYPT) en PHP), telephone (NULL possible)
Retour	@code (valeurs possibles : OK, EMAIL_DEJA_UTILISE, EMAIL_INVALIDE, MDP_TROP_FAIBLE) + @new_id (id du client créé)

Q-CLI-02 — Connexion : récupérer les identifiants d'un client

Champ	Valeur
Id	EF-VIS-07, RB-12, SEC-04/05
Écran	POST /connexion
Question	Récupérer l'id, le hash et l'état actif du client à partir de l'email saisi.
Type	CALL sp_get_credentials(:email, 'CLIENT') — puis vérifier actif = 1 en PHP
Retour	id_client, mot_de_passe_hash, actif — la comparaison du mot de passe se fait en PHP par password_verify(), jamais en SQL
Important : la même procédure accepte 'ADMIN' comme second paramètre pour le back-office.	

Q-CLI-03 — Récupérer les informations d'un client pour la page "Mon compte"

Champ	Valeur
Id	EF-CLI-01
Écran	GET /compte
Question	Afficher le formulaire pré-rempli avec nom, prénom, email, téléphone, adresse, CP, ville.
Type	SELECT
Tables	client
Filtres	WHERE id_client = :id (id depuis la session)
SEC-08 : jamais on ne passe l'id par la requête HTTP sur cette page ; on le lit depuis \$_SESSION['user_id']	

```
SELECT id_client, nom, prenom, email, telephone,
       adresse_livraison, code_postal, ville, actif, date_creation
FROM client
WHERE id_client = :id
LIMIT 1;
```

Q-CLI-04 — Modifier les informations d'un client

Champ	Valeur
Id	EF-CLI-01

Écran	POST /compte
Question	Mettre à jour nom, prénom, email, téléphone, adresse, CP, ville.
Type	CALL sp_update_client(...) — ne pas faire d'UPDATE direct
Retour	@code = OK ou EMAIL_DEJA_UTILISE

Q-CLI-05 — Changer le mot de passe d'un client (vérification ancien mdp)

Champ	Valeur
Id	EF-CLI-02, RB-12
Écran	POST /compte/mot-de-passe
Question	1) Vérifier l'ancien mot de passe en relisant le hash ; 2) si OK, mettre à jour le hash.
Type	1) SELECT mot_de_passe_hash FROM client WHERE id_client = :id ; 2) password_verify() en PHP ; 3) UPDATE client SET mot_de_passe_hash = :hash WHERE id_client = :id
Important : cette opération n'a pas de procédure dédiée. Régénérer l'ID de session (session_regenerate_id(true)) après succès.	

```
-- Étape 1 (lecture)
SELECT mot_de_passe_hash FROM client WHERE id_client = :id LIMIT 1;
-- Étape 3 (écriture, après password_verify + password_hash en PHP)
UPDATE client
  SET mot_de_passe_hash = :hash
 WHERE id_client = :id;
```

2.4 Module Panier et Commande (UC-06, UC-07, UC-08, UC-09)

Le panier vit **en session PHP**, pas en base. Aucun **INSERT/UPDATE** sur une table "panier". Les requêtes SQL de ce module concernent :

- la lecture des produits du panier pour recalculer les prix **depuis la base** (pas depuis le navigateur),
- l'appel à la procédure transactionnelle de création de commande,
- la lecture de l'historique et du détail des commandes.

Q-PAN-01 — Réserver les données produits du panier pour affichage/recalcul

Champ	Valeur
Id	EF-CLI-06, EF-GEN-01, RB-15/16
Écran	GET /panier

Question	Pour chaque id_produit dans le panier session, récupérer le prix TTC courant, le nom, le slug, le stock et la visibilité. Les totaux (sous-total HT, TVA, total TTC) sont recalculés côté serveur à partir de ces données, pas à partir des prix stockés dans la session client.
Type	SELECT ... WHERE id_produit IN (...)
Tables	produit
Filtres	WHERE id_produit IN (?, ?, ?, ?) (liste d'ids issue de la session)
Gestion des cas limites : un produit devenu visible=0 ou qui n'existe plus doit être retiré du panier (et signalé) ; une quantité supérieure au stock doit être écrêtée au stock courant côté PHP (message STOCK_INSUFFISANT).	

```
SELECT id_produit, nom, slug, prix_ht, tva, prix_ttc, stock, visible, image_url
FROM produit
WHERE id_produit IN (:id1, :id2, :id3 /* ... */);
```

PDO ne permet pas de binder directement un tableau : on génère autant de `?` que d'éléments du panier

(avec une limite de sécurité à ~100 pour éviter les abus).

Q-ORD-01 — Calcul des frais de port avant validation (aperçu)

Champ	Valeur
Id	EF-CLI-06, RB-15
Écran	page panier
Question	Quel serait le montant des frais de port pour un panier d'un montant TTC donné ? (Si le total $\geq$ franchise_port, port = 0 ; sinon port = frais_port.)
Type	SELECT calculatoire (pas d'UPDATE pour l'aperçu)
Tables	parametre

```
SELECT CASE
    WHEN :total_ttc >= (SELECT CAST(valeur AS DECIMAL(10,2))
                        FROM parametre WHERE cle = 'franchise_port')
    THEN 0.00
    ELSE (SELECT CAST(valeur AS DECIMAL(10,2))
          FROM parametre WHERE cle = 'frais_port')
END AS frais_port_estime;
```

Q-ORD-02 — Créer la commande depuis le panier (transaction complète)

Champ	Valeur
Id	EF-CLI-07, RB-04/05/06/18
Écran	POST /commande/valider

Question	Créer une commande ENTIÈREMENT par procédure : insert de la commande, insert des lignes, décrément du stock par trigger, calcul des frais de port, mise à jour du montant_total.
Type	CALL sp_create_order_from_basket(...) + CALL sp_confirm_order(...) en seconde étape, dans une seule transaction PDO (\$pdo->beginTransaction() / commit() / rollBack() en cas d'erreur).
Paramètres	id_client (session), adresse_livraison, lignes_json (chaîne JSON du tableau [{id_produit, quantite}, ...])
Retour	@code parmi : OK, STOCK_INSUFFISANT, PANIER_VIDE, PRODUIT_INEXISTANT, ADRESSE_MANQUANTE ; (id_commande créé), @numero (numéro

@new_cid

CMD2026-NNNNNN)

Appel type :

```
CALL sp_create_order_from_basket(:client, :adresse, :json);
-- puis si @code = 'OK' :
CALL sp_confirm_order(@new_cid);
-- et enfin COMMIT (via PDO)
```

Attention : **Piège classique** : la procédure effectue elle-même des COMMIT/ROLLBACK internes ?
Non — elle

signale les erreurs via `@code` et l'application décide du rollback (c'est pourquoi les tests T-29

vérifient que le stock revient en cas d'erreur). Vérifiez le comportement exact en lisant le

corps de `sp_create_order_from_basket` dans `sql/02_minishop_procedures.sql`.

Q-ORD-03 — Historique des commandes d'un client

Champ	Valeur
Id	EF-CLI-08, RB-08, SEC-08
Écran	GET /mes-commandes
Question	Afficher la liste paginée des commandes du client connecté, avec date, numéro, statut, montant total, frais de port, nombre d'articles.
Type	SELECT sur vue v_commandes_client
Filtres	OBLIGATOIRE : id_client = :id où :id vient de \$_SESSION['user_id']. Jamais confier l'id à l'URL sur cette page.
Tri	ORDER BY date_commande DESC
Pagination	LIMIT/OFFSET (10 par défaut, max 50)
Retour	colonnes de v_commandes_client

```
SELECT id_commande, numero, statut, montant_total, frais_port,
       date_commande, nb_lignes
FROM v_commandes_client
WHERE id_client = :id
ORDER BY date_commande DESC
LIMIT :limit OFFSET :offset;
```

Q-ORD-04 — Détail d'une commande (lignes + prix payés)

Champ	Valeur
Id	EF-CLI-09, RB-06, SEC-08
Écran	GET /mes-commandes/{id}
Question	Afficher le détail d'une commande appartenant au client connecté : adresse, montants, lignes avec le snapshot du prix au moment de l'achat.
Type	2 SELECT successifs
Filtres	systématiquement WHERE id_commande = :cid AND id_client = :id (jointure avec commande pour les lignes) — si ça ne ramène rien → 404

```
-- Entête de commande (contrôle d'appartenance dans la clause WHERE)
SELECT id_commande, numero, statut, montant_total, frais_port,
       adresse_livraison, commentaire, date_commande, date_modification
FROM commande
WHERE id_commande = :cid AND id_client = :id
LIMIT 1;

-- Lignes de la commande (prix déjà figés par trigger trg_ligne_prix_snapshot)
SELECT l.id_ligne, l.id_produit, p.nom, p.slug, p.image_url,
       l.quantite, l.prix_unitaire, l.total_ligne
FROM ligne_commande l
JOIN produit p ON p.id_produit = l.id_produit
WHERE l.id_commande = :cid
ORDER BY l.id_ligne ASC;
```

Q-ORD-05 — Historique des statuts (piste d'audit) côté client

Champ	Valeur
Id	EF-CLI-09
Écran	détail de commande
Question	Afficher la chronologie des statuts de la commande.
Type	SELECT sur order_status_history en filtrant sur l'id_commande après avoir vérifié l'appartenance (Q-ORD-04 a déjà ramené l'entête).

```
SELECT old_status, new_status, changed_by_role, commentaire, changed_at
FROM order_status_history
WHERE order_id = :cid
ORDER BY changed_at ASC, id ASC;
```

Q-ORD-06 — Annuler une commande

Champ	Valeur
Id	EF-CLI-10, RB-11/18/20
Écran	POST /mes-commandes/{id}/annulation
Question	Annuler une commande si elle n'est ni EXPEDIEE ni LIVREE. La procédure remet le stock par trigger, passe le statut à ANNULEE, remet frais_port et montant_total à 0 et écrit dans l'historique.

Type	CALL sp_cancel_order(:cid, :client, :commentaire)
Retour	@code = OK ou COMMANDE_NON_ANNULABLE ou ACCES_NON_AUTORISE ou COMMANDE_INTROUVABLE

2.5 Module Back-office — Produits, catégories, stocks (UC-10, UC-11, UC-12)

Q-ADM-01 — Lister les produits (back-office, y compris masqués)

Champ	Valeur
Id	EF-ADM-01
Écran	GET /admin/produits
Question	Liste paginée de tous les produits (y compris visible = 0) avec catégorie, état de stock, prix TTC.
Type	SELECT — ne pas utiliser v_catalogue (qui exclut les masqués) ; faire la jointure à la main
Tables	produit, categorie, v_etat_stock
Filtres : recherche par nom OU référence LIKE, filtre par visible, filtre par catégorie, filtre par état (etat = 'RUPTURE', etc.)	
Tri : liste blanche nom, prix_ttc, stock, date_modification	

```
SELECT p.id_produit, p.reference, p.slug, p.nom, p.prix_ht, p.tva, p.prix_ttc,
       p.stock, p.visible, p.date_modification,
       c.nom AS categorie, e.etat
FROM produit p
JOIN categorie c    ON c.id_categorie = p.id_categorie
JOIN v_etat_stock e ON e.id_produit  = p.id_produit
WHERE (:q IS NULL OR p.nom LIKE :qlike OR p.reference LIKE :qlike)
      AND (:cat IS NULL OR p.id_categorie = :cat)
      AND (:vis IS NULL OR p.visible = :vis)
ORDER BY p.date_modification DESC
LIMIT :limit OFFSET :offset;
```

(puis un `SELECT COUNT(*)` calqué sur les mêmes filtres pour la pagination.)

Q-ADM-02 — Charger un produit pour le formulaire d'édition

Champ	Valeur
Id	EF-ADM-01
Écran	GET /admin/produits/{id}/editer
Question	Toutes les colonnes d'un produit pour pré-remplir le formulaire (y compris visible = 0).
Type	SELECT produit par id

```
SELECT p.*, c.nom AS categorie
FROM produit p
```

```
JOIN categorie c ON c.id_categorie = p.id_categorie
WHERE p.id_produit = :id
LIMIT 1;
```

Q-ADM-03 — Créer / modifier un produit

Champ	Valeur
Id	EF-ADM-01, RB-02/07/14/16
Écran	POST /admin/produits
Question	Création ou mise à jour d'un produit (la procédure gère l'INSERT ou l'UPDATE selon que p_id_produit est fourni).
Type	CALL sp_save_product(...)
Retour	@code = OK / REFERENCE_DEJA_UTILISEE / SLUG_DEJA_UTILISE / RB02_PRIX_DOIT_ETRE_STRICTEMENT_POSITIF / CATEGORIE_INTROUVABLE, et @new_id (id du produit)

Q-ADM-04 — Supprimer (ou masquer) un produit

Champ	Valeur
Id	EF-ADM-02, RB-09/14
Écran	POST /admin/produits/{id}/suppression
Question	Supprimer un produit s'il n'a jamais été commandé (DELETE physique), sinon le basculer en visible = 0. La procédure sp_delete_product gère les deux cas.
Type	CALL sp_delete_product(:id) — ne jamais faire de DELETE FROM produit direct (le trigger trg_produit_delete l'interdirait si le produit est référencé)

Q-ADM-05 — Liste des catégories

Champ	Valeur
Id	EF-ADM-03
Écran	GET /admin/categories
Question	Afficher toutes les catégories avec le nombre de produits (même masqués) qu'elles contiennent.
Type	SELECT avec JOIN + COUNT
Différence avec Q-COM-02 : compter tous les produits, pas seulement les visibles	

```
SELECT c.id_categorie, c.nom, c.slug, c.description,
       COUNT(p.id_produit) AS nb_produits
FROM categorie c
LEFT JOIN produit p ON p.id_categorie = c.id_categorie
GROUP BY c.id_categorie, c.nom, c.slug, c.description
ORDER BY c.nom ASC;
```

Q-ADM-06 — Charger une catégorie pour l'édition

Champ	Valeur
Id	EF-ADM-03
Écran	GET /admin/categories/{id}/editer
Question	Une catégorie par id.
Type	SELECT * FROM categorie WHERE id_categorie = :id LIMIT 1;

Q-ADM-07 — Créer/modifier/supprimer une catégorie

Champ	Valeur
Id	EF-ADM-03, RB-07/08/14
Écran	POST /admin/categories et POST /admin/categories/{id}/suppression
Question	CRUD catégorie.
Type	CALL sp_save_category(...) / CALL sp_delete_category(:id) — la suppression lève CATEGORIE_NON_VIDE_REASSIGNER_LES_PRODUITS si des produits y sont rattachés.

Q-ADM-08 — Liste des catégories pour le `` du formulaire produit

Champ	Valeur
Id	EF-ADM-01
Écran	formulaire création/édition produit
Question	Fournir la liste complète des catégories (id, nom) pour peupler un <code><select></code> .
Type	SELECT id_categorie, nom FROM categorie ORDER BY nom ASC;

Q-ADM-09 — Ajuster le stock

Champ	Valeur
Id	EF-ADM-04, RB-03/17/18
Écran	POST /admin/stocks
Question	Ajouter/retirer du stock (mode DELTA) ou fixer sa valeur absolue (mode SET), avec motif.
Type	CALL sp_adjust_stock(:id, :mode, :qte, :motif, :admin_id) — la procédure pose un verrou (FOR UPDATE) et lève

RB03_STOCK_NE_PEUT_PAS_ETRE_NEGATIF

Q-ADM-10 — Alertes de stock (ruptures et seuils)

Champ	Valeur
Id	EF-ADM-05, ENF-13
Écran	GET /admin/stocks?etat=alerte
Question	Liste des produits dont l'état est RUPTURE ou TRES_BAS, triés par criticité (rupture d'abord, puis stock croissant).

Type	SELECT sur v_etat_stock JOIN produit JOIN categorie
Filtres	WHERE e.etat IN ('RUPTURE','TRES_BAS')
Tri	ORDER BY FIELD(e.etat, 'RUPTURE','TRES_BAS'), e.stock ASC

```
SELECT e.id_produit, p.reference, p.nom, e.stock, e.seuil_alerte, e.etat,
       c.nom AS categorie
FROM v_etat_stock e
JOIN produit p ON p.id_produit = e.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
WHERE e.etat IN ('RUPTURE','TRES_BAS')
ORDER BY FIELD(e.etat, 'RUPTURE','TRES_BAS'), e.stock ASC;
```

2.6 Module Back-office — Commandes, statuts, audit, indicateurs (UC-13, UC-14)

Q-ADM-11 — Liste des commandes (back-office)

Champ	Valeur
Id	EF-ADM-06
Écran	GET /admin/commandes
Question	Lister les commandes avec possibilité de filtrer par statut, client (id/email), période (date_min, date_max), montant min/max.
Type	SELECT sur v_commandes_client (déjà jointe au client) + WHERE dynamique
Tri : liste blanche numero, date_commande, montant_total, client ; par défaut date_commande DESC	
Pagination : 20 par page, max 100	
Bonus : renvoyer également le SUM(montant_total) des lignes filtrées (utile à l'admin) via une seconde requête SELECT SUM(...) avec les mêmes filtres.	

```
SELECT id_commande, numero, client, id_client, statut, montant_total,
       frais_port, date_commande, nb_lignes
FROM v_commandes_client
WHERE (:statut IS NULL OR statut = :statut)
AND (:deb IS NULL OR date_commande >= :deb)
AND (:fin IS NULL OR date_commande <= :fin)
AND (:client IS NULL
      OR id_client = :client_id
      OR client LIKE :client_like)
ORDER BY date_commande DESC
LIMIT :limit OFFSET :offset;
```

Q-ADM-12 — Détail d'une commande côté admin

Champ	Valeur
Id	EF-ADM-08
Écran	GET /admin/commandes/{id}
Question	Entête complet de la commande, les lignes (prix snapshot), le client (toutes ses coordonnées), la piste d'audit.

| **Type** : plusieurs **SELECT** (pas de contrôle d'appartenance — l'admin voit tout) :

1. entête : **commande** JOIN **client** par **id_commande**
2. lignes : identique à Q-ORD-04 (seconde requête)
3. historique : identique à Q-ORD-05 |

```
-- 1. Entête + client
SELECT o.*, cl.nom, cl.prenom, cl.email, cl.telephone,
       cl.adresse_livraison AS cl_adr, cl.code_postal AS cl_cp, cl.ville AS cl_ville
FROM commande o
JOIN client cl ON cl.id_client = o.id_client
WHERE o.id_commande = :cid
LIMIT 1;

-- 2. Lignes : identique à Q-ORD-04 (ligne_commande JOIN produit)
-- 3. Historique : identique à Q-ORD-05 (order_status_history par order_id)
```

Q-ADM-13 — Modifier le statut d'une commande

Champ	Valeur
Id	EF-ADM-07, RB-09/11
Écran	POST /admin/commandes/{id}/statut
Question	Passer une commande à un nouveau statut avec commentaire (commentaire obligatoire quand le statut cible est ANNULEE).
Type	CALL sp_update_order_status(:cid, :new_status, :admin_id, 'ADMIN', :commentaire) — la procédure + le trigger trg_commande_transition_statut vérifient la matrice RB-11 et lèvent RB11_TRANSITION_STATUT_INTERDITE si la transition est interdite

Q-ADM-14 — Piste d'audit des statuts (back-office)

Champ	Valeur
Id	EF-ADM-08
Question	Même requête que Q-ORD-05 mais en affichant en plus l'identité de l'auteur (nom/prenom si admin, "Le client" si client). Cela requiert une jointure spécifique parce que changed_by pointe soit vers client soit vers administrateur selon changed_by_role.
Type	SELECT avec deux LEFT JOIN

```
SELECT h.old_status, h.new_status, h.changed_by_role, h.commentaire, h.changed_at,
```

```

CASE h.changed_by_role
  WHEN 'CLIENT' THEN CONCAT(cl.prenom, ' ', cl.nom)
  WHEN 'ADMIN' THEN CONCAT(ad.prenom, ' ', ad.nom, ' (', ad.role, ')')
  ELSE 'Système'
END AS auteur
FROM order_status_history h
LEFT JOIN client cl      ON h.changed_by_role = 'CLIENT'
                        AND cl.id_client = h.changed_by
LEFT JOIN administrateur ad ON h.changed_by_role = 'ADMIN'
                        AND ad.id_admin = h.changed_by
WHERE h.order_id = :cid
ORDER BY h.changed_at ASC, h.id ASC;

```

Q-ADM-15 — Indicateurs du tableau de bord

Champ	Valeur
Id	EF-ADM-09
Écran	GET /admin
Question	CA encaissé du jour, CA par statut, panier moyen, top 5 des produits les plus vendus, nombre de commandes en attente, nombre de produits en rupture.

| Type :

- CA par statut, panier moyen, top produits → ``CALL sp_revenue_report(:deb, :fin)`` (la procédure retourne plusieurs jeux de résultats)
- Compteurs rapides → `SELECT COUNT/SUM` directs |

Compteurs rapides à écrire en direct :

```

-- Nombre de commandes en attente (EN_PREPARATION)
SELECT COUNT(*) AS nb_en_preparation
  FROM commande WHERE statut = 'EN_PREPARATION';

-- Nombre de produits en rupture
SELECT COUNT(*) AS nb_ruptures
  FROM v_etat_stock WHERE etat = 'RUPTURE';

-- CA du jour
SELECT COALESCE(SUM(montant_total), 0) AS ca_jour
  FROM commande
 WHERE statut IN ('PAYEE', 'EXPEDIEE', 'LIVREE')
    AND DATE(date_commande) = CURRENT_DATE;

```

2.7 Module Back-office — Gestion des comptes administrateurs (SUPER only)

Q-ADM-16 — Liste des administrateurs

Champ	Valeur
-------	--------

Id	EF-ADM-10, RB-12/13
Écran	GET /admin/equipe (rôle SUPER uniquement, contrôlé en PHP)
Question	Tous les administrateurs avec leur rôle et état.
Type	SELECT id_admin, nom, prenom, email, role, actif, derniere_connexion FROM administrateur ORDER BY nom, prenom;

Q-ADM-17 — Créer un administrateur

Champ	Valeur
Id	EF-ADM-10
Écran	POST /admin/equipe
Question	Créer un nouveau compte admin. Aucune procédure n'existe pour cela — INSERT direct, avec hash généré par password_hash() en PHP.
Type	INSERT INTO administrateur(...) VALUES(...)

```
INSERT INTO administrateur (nom, prenom, email, mot_de_passe_hash, role, actif)
VALUES (:nom, :prenom, :email, :hash, :role, 1);
```

Q-ADM-18 — Bloquer / débloquent un administrateur

Champ	Valeur
Id	EF-ADM-10
Question	Passer actif de 0 à 1 ou inversement.
Type	UPDATE administrateur SET actif = :actif WHERE id_admin = :id;
Sécurité : interdire à un admin de se bloquer lui-même.	

Q-ADM-19 — Changer le rôle d'un administrateur

Champ	Valeur
Id	EF-ADM-10
Question	Passer un compte de GESTIONNAIRE à SUPER et vice-versa.
Type	UPDATE administrateur SET role = :role WHERE id_admin = :id; — le nouveau rôle doit être validé par la liste blanche ('SUPER','GESTIONNAIRE') en PHP avant bind.

Q-ADM-20 — Connexion administrateur

Champ	Valeur
Id	EF-ADM-10, SEC-04
Écran	POST /admin/connexion
Question	Même logique que Q-CLI-02 mais pour le back-office.

Type	CALL sp_get_credentials(:email, 'ADMIN') → vérification actif=1 → password_verify() en PHP → mise à jour de derniere_connexion (Q-COM-05) → session_regenerate_id(true).
------	--

2.8 Module Sécurité / CSRF / journalisation

Q-SEC-01 — Lire un utilisateur par son token "se souvenir de moi" (SI implémenté)

Non prévu par le CDC → à ne pas implémenter.

Q-SEC-02 — Journal des rejets de règles (optionnel EF-GEN-04)

Champ	Valeur
Id	EF-GEN-04, SEC-13
Question	Enregistrer les rejets (tentatives de suppression d'un produit référencé, IDOR, CSRF, etc.) dans une table de log.
Type	La table de log n'est pas encore dans le schéma actuel. Soit vous la créez (migration à faire), soit vous utilisez le fichier app/var/log/app.log via Monolog/error_log. Cette requête n'est pas à écrire tant que la table n'existe pas — elle devra être ajoutée dans un script de migration.

2.9 Installateur / CI / scripts

Q-INST-01 — Vérifier que la base est bien initialisée (install.php)

Champ	Valeur
Id	EF-GEN-06
Écran	public/install.php
Question	Vérifier que les 8 tables existent et qu'il y a au moins un administrateur et un client.
Type	SHOW TABLES LIKE 'client'; ou SELECT COUNT(*) FROM administrateur; — pas de requête métier particulière, c'est un simple contrôle.

Q-INST-02 — Vérifier que l'administrateur de seed est présent (CI)

Champ	Valeur
Écran	.gitlab-ci.yml job seed
Question	Compter les lignes insérées dans chaque table après scripts/load_db.sh.

Type	SELECT COUNT(*) FROM <table>; sur les 8 tables — déjà implémenté dans load_db.sh (à terminer, cf. tâche D3-L5-05 dans la todolist).
------	---

3. Récapitulatif des requêtes à écrire (check-list pour la collègue)

Id	Écran	Procédure à appeler	Où requête à écrire	Fait [OK]
Q-COM-01	tous	—	SELECT valeur FROM parametre ou fn_param()	[]
Q-COM-02	menu catégories	—	categorie LEFT JOIN produit WHERE visible=1 COUNT	[]
Q-COM-03	auth admin	—	SELECT ... FROM administrateur WHERE id=? AND actif=1	[]
Q-COM-04	connexion client	—	UPDATE client SET derniere_connexion = ...	[]
Q-COM-05	connexion admin	—	UPDATE administrateur SET derniere_connexion = ...	[]
Q-VIS-01	/catalogue	—	SELECT ... FROM v_catalogue ORDER BY ... LIMIT ? OFFSET ?	[]
Q-VIS-02	catalogue (pagination)	—	SELECT COUNT(*) FROM v_catalogue (+ filtres)	[]
Q-VIS-03	catalogue filtré	(facultatif sp_search_products(q=NULL, ...))	SELECT sur v_catalogue avec WHERE dynamique	[]
Q-VIS-04	/recherche	sp_search_products	— (appel seulement)	[]
Q-VIS-05	/produit/{slug}	—	SELECT ... FROM v_catalogue WHERE slug=?	[]
Q-VIS-06	/categorie/{slug}	—	Q-VIS-03 avec categorie_slug=?	[]
Q-VIS-07	ajouter panier	—	SELECT stock, visible FROM produit WHERE id=?	[]
Q-CLI-01	/inscription	sp_create_account	—	[]
Q-CLI-02	/connexion (client)	sp_get_credentials(email, 'CLIENT')	—	[]
Q-CLI-03	/compte	—	SELECT ... FROM client WHERE id=?	[]
Q-CLI-04	POST /compte	sp_update_client	—	[]

Q-CLI-05	mdp oublié/changé	—	SELECT hash puis UPDATE hash	[]
Q-PAN-01	/panier	—	SELECT ... FROM produit WHERE id IN (?)	[]
Q-ORD-01	panier (aperçu port)	—	SELECT CASE ... (calcul frais de port)	[]
Q-ORD-02	POST /commande/valider	sp_create_order_from _basket + sp_confirm_order	— (transaction PDO)	[]
Q-ORD-03	/mes-commandes	—	SELECT ... FROM v_commandes_client WHERE id_client=?	[]
Q-ORD-04	détail commande client	—	SELECT entête + SELECT lignes	[]
Q-ORD-05	détail commande (historique)	—	SELECT sur order_status_history	[]
Q-ORD-06	annulation	sp_cancel_order	—	[]
Q-ADM-01	/admin/produits	—	SELECT produit + categorie + v_etat_stock	[]
Q-ADM-02	édition produit	—	SELECT produit WHERE id=?	[]
Q-ADM-03	POST produit	sp_save_product	—	[]
Q-ADM-04	suppression produit	sp_delete_product	—	[]
Q-ADM-05	/admin/categories	—	SELECT categorie LEFT JOIN produit COUNT	[]
Q-ADM-06	édition catégorie	—	SELECT categorie WHERE id=?	[]
Q-ADM-07	CRUD catégorie	sp_save_category / sp_delete_category	—	[]
Q-ADM-08	select catégories (form)	—	SELECT id, nom FROM categorie ORDER BY nom	[]
Q-ADM-09	stocks	sp_adjust_stock	—	[]
Q-ADM-10	alertes stocks	—	SELECT sur v_etat_stock WHERE etat IN ('RUPTURE', 'TRES_BAS')	[]
Q-ADM-11	/admin/commandes	—	SELECT sur v_commandes_client WHERE dynamique	[]
Q-ADM-12	détail commande admin	—	SELECT entête + lignes + historique (3 requêtes)	[]
Q-ADM-13	changement statut	sp_update_order_stat us	—	[]
Q-ADM-14	audit (admin)	—	SELECT history + LEFT JOIN client/admin	[]

Q-ADM-15	tableau de bord	sp_revenue_report + 3 SELECT directs (compteurs)	—	[]
Q-ADM-16	/admin/equipe	—	SELECT administrateur ORDER BY nom	[]
Q-ADM-17	créer admin	—	INSERT INTO administrateur(...)	[]
Q-ADM-18	bloquer/débloquer admin	—	UPDATE administrateur SET actif=?	[]
Q-ADM-19	changer rôle admin	—	UPDATE administrateur SET role=?	[]
Q-ADM-20	connexion admin	sp_get_credentials(e mail, 'ADMIN')	—	[]
Q-INST-01	install.php	—	SHOW TABLES / comptages	[]
Q-INST-02	CI seed	—	SELECT COUNT(*) FROM <table> (dans load_db.sh)	[]

Total : 43 requêtes à écrire / appels de procédures à câbler.

4. Bonnes pratiques et pièges à éviter

1. **Ne jamais réécrire les règles métier en PHP** : `frais_port`, calcul du TTC, décrémentation de stock,

matrice de statuts, suppression logique, tout passe par les procédures et triggers.

1. **Jamais de `SELECT \*` en production** dans les écrans : ne sélectionner que les colonnes utiles

(j'ai mis `*` dans certains exemples pour simplifier, mais dans le code final listez explicitement les colonnes nécessaires à la vue).

1. **Ne jamais faire de DELETE / UPDATE sur `commande` ou `ligne\_commande` en direct côté client** :

les triggers interdiront les montants calculés (RB-15) et les transitions de statut interdites (RB-11), mais c'est une erreur de conception de tenter de les contourner.

1. **Transactions** : seule `sp_create_order_from_basket` + `sp_confirm_order` nécessitent une transaction

applicative explicite. Toutes les autres procédures sont autonomes.

1. **Pagination** : toujours exécuter le `COUNT(*)` avant le `SELECT ... LIMIT` avec les **mêmes filtres**, sinon le nombre de pages sera faux.

1. **Tests** : les requêtes écrites doivent faire passer les tests `tests/sql/tNN_*.sql` (29 tests) et les cas fonctionnels F-01...F-31, ainsi que les contrôles S-01...S-08 (cf. `tests/security/cont`

`roles.sh` à terminer par A).

1. **Lecture des codes retour** : toutes les procédures renvoient un `@code` de type VARCHAR. Toujours le récupérer et le mapper vers un message utilisateur traduit (table de correspondance dans le code PHP : `OK`, `STOCK_INSUFFISANT`, `EMAIL_DEJA_UTILISE`, `COMMANDE_NON_ANNULABLE`, `RB11_TRANSITION_STATUT_INTERDITE`, `RB03_STOCK_NE_PEUT_PAS_ETRE_NEGATIF`, `REFERENCE_DEJA_UTILISEE`, `SLUG_DEJA_UTILISE`, `CATEGORIE_NON_VIDE`, `CATEGORIE_INTROUVABLE`, `PRODUIT_REFERENCE_INTERDIT_DE_SUPPRIMER`, `ACCES_NON AUTORISE`, `PANIER_VIDE`, `ADRESSE_MANQUANTE`, `PRODUIT_INEXISTANT`, `COMMANDE_INTROUVABLE`, `RB02_PRIX_DOIT_ETRE_STRICTEMENT_POSITIF`).

5. Références

- Schéma complet : `sql/01_minishop_schema.sql`
- Procédures : `sql/02_minishop_procedures.sql`
- Triggers : `sql/03_minishop_triggers.sql`
- Données de démo : `sql/04_minishop_demo.sql`
- Exigences EF-/SEC-/RB-* : `docs/01-cahier-des-charges-MiniShop.md` §4
- WBS : `docs/05-wbs-projet.md`
- Todolist : `docs/06-todolist.md` (v1.1 — 2 développeurs)
- Tests SQL : `tests/sql/` (29 tests rejouables via `./scripts/run_sql_tests.sh`)